

Run the below command in the terminal to start the vLLM server

```
VLLM_USE_TRITON_FLASH_ATTN=0 \
vllm serve Qwen/Qwen3-4B \
--served-model-name Qwen3-4B \
--api-key abc-123 \
--port 8000 \
--enable-auto-tool-choice \
--tool-call-parser hermes \
--trust-remote-code \
--max_model_len 24272
```

1. Setup

In [1]: `pip install -q matplotlib scikit-learn langchain`

```
[33mWARNING: Running pip as the 'root' user can result in broken
permissions and conflicting behaviour with the system package
manager, possibly rendering your system unusable. It is recommended
to use a virtual environment instead:
https://pip.pypa.io/warnings/venv. Use the --root-user-action option
if you know what you are doing and want to suppress this
warning.[0m[33m
[0m
[1m[0m[34;49mnotice[0m[1;39;49m][0m[39;49m A new release of
pip is available: [0m[31;49m25.2[0m[39;49m ->
[0m[32;49m26.1.2[0m
[1m[0m[34;49mnotice[0m[1;39;49m][0m[39;49m To update, run:
[0m[32;49mpython3.12 -m pip install --upgrade pip[0m
Note: you may need to restart the kernel to use updated packages.
```

```
In [24]: from __future__ import annotations

import json, re, uuid, random, os, warnings, textwrap, threading, time
from datetime import datetime, timedelta
from typing import TypedDict, Annotated, Sequence, Optional, Any, Dict, List
import operator

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, HTML, Markdown, clear_output
import ipywidgets as widgets
from langchain_core.messages import AIMessage

warnings.filterwarnings("ignore")
random.seed(42)
np.random.seed(42)

# Plotting style – dark cyberpunk theme
plt.rcParams.update({
    "figure.facecolor": "#020608",
    "axes.facecolor": "#081520",
    "axes.edgecolor": "#0d3348",
    "axes.labelcolor": "#c8dde8",
    "text.color": "#c8dde8",
    "xtick.color": "#4a7a90",
    "ytick.color": "#4a7a90",
    "grid.color": "#0d3348",
    "grid.linestyle": "--",
    "grid.alpha": 0.5,
    "legend.facecolor": "#081520",
    "legend.edgecolor": "#0d3348",
    "font.family": "monospace",
})

print("Imports complete")
```

Imports complete

```
In [25]: VLLM_MODEL      = "Qwen3-4B"
VLLM_BASE_URL  = "http://localhost:8000/v1"
VLLM_API_KEY   = "abc-123"
```

2. Synthetic Dataset

```
In [26]: # Helper utilities
def _ts(days_ago_max: int = 60) -> str:
    delta = timedelta(
        days=random.randint(0, days_ago_max),
        hours=random.randint(0, 23),
        minutes=random.randint(0, 59),
        seconds=random.randint(0, 59),
    )
    return (datetime.utcnow() - delta).isoformat() + "Z"

def _ip(private: bool = False) -> str:
    if private:
        return f"192.168.{random.randint(1,254)}.{random.randint(1,254)}"
    return f"{random.randint(1,223)}.{random.randint(0,255)}.{random.randint(0,255)}.{random.randint(1,254)}"

_USERS =
["alice.chen","bob.martinez","carol.wu","david.singh","eve.johnson",
 "frank.lee","grace.kim","henry.patel","iris.brown","jake.davis",

 "karen.white","liam.garcia","maya.robinson","noah.miller","olivia.taylor"]
_HOSTS = lambda: f"
{random.choice(['WS','SRV','DB','APP','WEB','DMZ','PROX','FW'])}-
{random.randint(100,999)}"

SEVERITIES = ["CRITICAL","HIGH","MEDIUM","LOW","INFO"]
SEV_W      = [0.10, 0.25, 0.35, 0.20, 0.10]
STATUSES   =
["OPEN","IN_PROGRESS","CONTAINED","RESOLVED","FALSE_POSITIVE"]
STAT_W     = [0.30, 0.25, 0.15, 0.25, 0.05]
_sev       = lambda: random.choices(SEVERITIES, SEV_W)[0]
_stat      = lambda: random.choices(STATUSES, STAT_W)[0]

print("Helpers ready")
```

Helpers ready

In [27]:

```

# Event generators
def _network_events(n: int = 60) -> list:
    types = ["Port Scan","SYN Flood","DNS Tunneling","Lateral Movement",
             "Beaconing","Data Exfiltration","ARP Spoofing","ICMP Sweep",
             "SMB Brute Force","RDP Brute Force"]
    return [{
        "id": str(uuid.uuid4()), "category": "NETWORK", "type":
random.choice(types),
        "timestamp": _ts(), "severity": _sev(), "status": _stat(),
        "source_ip": _ip(), "dest_ip": _ip(private=True),
        "dest_port":
random.choice([22,80,443,445,3389,8080,8443,53,21,3306]),
        "protocol":
random.choice(["TCP","UDP","ICMP","DNS","HTTP","HTTPS","SMB","RDP"]),
        "bytes_transferred": random.randint(512, 50_000_000),
        "packet_count": random.randint(10, 100_000),
        "confidence_score": round(random.uniform(0.55, 0.99), 2),
        "detected_by": "ThreatDetectionAgent",
        "ml_model": random.choice(["IsolationForest","LSTM-
Autoencoder","CNN-Flow"]),
        "description": f"Suspicious network activity detected from external
source",
        "mitre_tactic":
random.choice(["TA0001","TA0002","TA0006","TA0007","TA0008","TA0010","TA001
1"]),
        "mitre_technique": f"T{random.randint(1000,1999)}",
        "false_positive_prob": round(random.uniform(0.01, 0.25), 3),
        "recommended_action": random.choice(["Block source IP","Isolate
endpoint","Increase monitoring","Alert SOC","Auto-contained"]),
    } for _ in range(n)]

def _malware_events(n: int = 55) -> list:
    families = ["Emotet","Ryuk","TrickBot","Cobalt
Strike","Mimikatz","Raccoon Stealer",
               "BlackCat","LockBit","Conti","AsyncRAT","Agent
Tesla","RedLine","Qbot","Dridex","Njrat"]
    types =
["Ransomware","Trojan","Spyware","Backdoor","RAT","Rootkit","Wiper","Droppe
r","Loader","Keylogger"]
    return [{
        "id": str(uuid.uuid4()), "category": "MALWARE", "type":
random.choice(types),
        "malware_family": random.choice(families), "timestamp": _ts(),
        "severity": _sev(), "status": _stat(), "affected_host": _HOSTS(),
        "file_hash_md5": uuid.uuid4().hex,
        "file_hash_sha256": uuid.uuid4().hex + uuid.uuid4().hex,
        "file_name":
random.choice(["update.exe","invoice.pdf.exe","setup_v2.msi","svchost32.exe",
"chrome_helper.exe"]),
        "file_size_bytes": random.randint(10_000, 5_000_000),
        "detection_method": random.choice(["Static Analysis","Dynamic
Sandbox","Signature Match","Behavior ML"]),
        "c2_server": f"{_ip()}:{random.choice([80,443,8080,4444,8888])}",

```



```

        "confidence_score": round(random.uniform(0.60, 0.99), 2),
        "detected_by": "MalwareAnalysisAgent",
        "ml_model": random.choice(["CNN-Binary", "GNN-CallGraph", "BERT-SecNLP"])),
        "sandbox_verdict":
random.choice(["MALICIOUS", "SUSPICIOUS", "CLEAN"]),
        "persistence_mechanism": random.choice(["Registry Run
Key", "Scheduled Task", "Service Install", "Startup Folder", "Boot Sector"]),
        "description": f"Malware detected on endpoint – immediate
containment recommended",
        "mitre_tactic":
random.choice(["TA0002", "TA0003", "TA0005", "TA0009", "TA0040"]),
        "mitre_technique": f"T{random.randint(1000,1999)}",
        "false_positive_prob": round(random.uniform(0.005, 0.15), 3),
        "recommended_action": random.choice(["Quarantine file", "Isolate
host", "Wipe and reimage", "Collect forensics", "Block hash"]),
    } for _ in range(n)]

def _identity_events(n: int = 55) -> list:
    actions = ["Failed Login", "Credential Stuffing", "Password Spray", "MFA
Bypass Attempt",
        "Privilege Escalation", "Lateral Movement", "Impossible
Travel", "Account Takeover",
        "Service Account Abuse", "Token Theft", "Kerberoasting", "Pass-
the-Hash"]
    return [{
        "id": str(uuid.uuid4()), "category": "IDENTITY", "type":
random.choice(actions),
        "timestamp": _ts(), "severity": _sev(), "status": _stat(),
        "username": random.choice(_USERS), "source_ip": _ip(),
        "target_host": _HOSTS(),
        "failed_attempts": random.randint(1, 500),
        "account_type":
random.choice(["Standard", "Admin", "Service", "Privileged"]),
        "auth_method":
random.choice(["Password", "SSO", "Kerberos", "NTLM", "Certificate", "Token"]),
        "risk_score": round(random.uniform(10, 100), 1),
        "location_country":
random.choice(["US", "CN", "RU", "BR", "IN", "DE", "UK", "UA", "KR", "IR"]),
        "user_behavior_baseline":
random.choice(["NORMAL", "ANOMALOUS", "HIGHLY_ANOMALOUS"]),
        "confidence_score": round(random.uniform(0.55, 0.99), 2),
        "detected_by": "IdentityAccessAgent",
        "ml_model": random.choice(["UEBA-Cluster", "GraphML-Identity", "LSTM-
Session"])),
        "description": f"Suspicious identity activity detected",
        "mitre_tactic":
random.choice(["TA0001", "TA0003", "TA0004", "TA0006", "TA0008"]),
        "mitre_technique": f"T{random.randint(1000,1999)}",
        "false_positive_prob": round(random.uniform(0.02, 0.30), 3),
        "recommended_action": random.choice(["Lock account", "Force
MFA", "Reset credentials", "Notify user", "Escalate to SOC"]),
    } for _ in range(n)]

```

```

def _incident_events(n: int = 50) -> list:
    types = ["Data Breach", "Ransomware Attack", "Business Email
Compromise", "Supply Chain Attack",
            "Insider Threat", "DDoS Attack", "Phishing Campaign", "SQL
Injection", "XSS Attack",
            "Zero-Day Exploitation", "API Abuse", "Cloud Misconfiguration"]
    return [{
        "id": str(uuid.uuid4()), "category": "INCIDENT", "type":
random.choice(types),
        "timestamp": _ts(), "severity": _sev(), "status": _stat(),
        "affected_systems": [_HOSTS() for _ in range(random.randint(1,
8))],
        "affected_users": [random.choice(_USERS) for _ in
range(random.randint(0, 5))],
        "estimated_data_loss_gb": round(random.uniform(0, 500), 2),
        "downtime_minutes": random.randint(0, 2880),
        "financial_impact_usd": random.randint(0, 5_000_000),
        "attack_vector":
random.choice(["Email", "Web", "Network", "Physical", "Insider", "Supply
Chain"]),
        "response_time_minutes": random.randint(1, 240),
        "containment_time_minutes": random.randint(5, 1440),
        "confidence_score": round(random.uniform(0.60, 0.99), 2),
        "detected_by": "IncidentResponseAgent",
        "playbook_executed": random.choice(["RANSOMWARE-001", "DATA-BREACH-
002", "DDOS-003", "PHISHING-004", "INSIDER-005"]),
        "description": f"Security incident – multi-team response
initiated",
        "mitre_tactic":
random.choice(["TA0040", "TA0010", "TA0009", "TA0006", "TA0011"]),
        "mitre_technique": f"T{random.randint(1000, 1999)}",
        "false_positive_prob": round(random.uniform(0.005, 0.10), 3),
        "recommended_action": random.choice(["Activate IR Team", "Executive
Notification", "Legal/Compliance Alert", "Forensic Preservation", "Isolate
Environment"]),
    } for _ in range(n)]

def _apt_events(n: int = 45) -> list:
    groups = ["APT28", "APT29", "APT41", "Lazarus
Group", "Sandworm", "Turla", "FIN7", "Carbanak",
            "UNC2452", "Scattered Spider", "ALPHV", "TA505"]
    campaigns = ["Operation ShadowStrike", "Campaign NightOwl", "Operation
IronGate",
                "Campaign DarkWater", "Operation CobaltFog", "Campaign
SilverThread"]
    return [{
        "id": str(uuid.uuid4()), "category": "APT", "type": "Advanced
Persistent Threat",
        "threat_group": random.choice(groups), "campaign_name":
random.choice(campaigns),
        "timestamp": _ts(), "severity": random.choices(["CRITICAL", "HIGH"],
[0.5, 0.5])[0],

```

```

        "status": _stat(),
        "attack_stage": random.choice(["Reconnaissance", "Initial
Access", "Execution", "Persistence",
                                     "Privilege Escalation", "Defense
Evasion", "Credential Access",
                                     "Discovery", "Lateral
Movement", "Collection", "Exfiltration", "C2"]]),
        "dwell_time_days": random.randint(1, 365),
        "affected_assets": [_HOSTS() for _ in range(random.randint(2,
15))],
        "iocs": {"ips": [_ip() for _ in range(3)], "domains": [f"evil-
{uuid.uuid4().hex[:8]}.com"], "hashes": [uuid.uuid4().hex]},
        "kill_chain_phase":
random.choice(["Delivery", "Exploitation", "Installation", "C2", "Actions on
Objectives"]),
        "confidence_score": round(random.uniform(0.65, 0.99), 2),
        "detected_by": "PredictiveIntelligenceAgent",
        "ml_model": random.choice(["GNN-APT", "XGBoost-
Campaign", "Transformer-IOC"]),
        "description": f"APT campaign indicators matched with high
confidence",
        "mitre_tactic":
random.choice(["TA0001", "TA0002", "TA0003", "TA0004", "TA0005", "TA0006", "TA000
7", "TA0008", "TA0009", "TA0010", "TA0011", "TA0040"]),
        "mitre_technique": f"T{random.randint(1000, 1999)}",
        "false_positive_prob": round(random.uniform(0.005, 0.12), 3),
        "recommended_action": random.choice(["Engage Threat Hunting", "CISA
Notification", "Emergency Patching", "Full IR Activation", "ISP Null Route"]),
    } for _ in range(n)]

def _vuln_events(n: int = 40) -> list:
    products = ["Apache Log4j", "OpenSSL", "Windows SMB", "Cisco IOS", "VMware
ESXi",
               "Fortinet VPN", "Exchange Server", "Spring Framework", "Linux
Kernel",
               "Atlassian Confluence", "Citrix ADC", "F5 BIG-IP", "MOVEit
Transfer"]
    results = []
    for _ in range(n):
        product = random.choice(products)
        cvss = round(random.uniform(3.0, 10.0), 1)
        sev = "CRITICAL" if cvss >= 9 else "HIGH" if cvss >= 7 else
"MEDIUM" if cvss >= 4 else "LOW"
        cve = f"CVE-{random.randint(2021, 2025)}-
{random.randint(10000, 99999)}"
        results.append({
            "id": str(uuid.uuid4()), "category": "VULNERABILITY", "type":
"CVE Exploitation",
            "cve_id": cve, "affected_product": product,
            "timestamp": _ts(), "severity": sev, "status": _stat(),
            "cvss_score": cvss, "exploit_available":
random.choice([True, False, True]),
            "exploit_in_wild": random.choice([True, False]),

```

```

        "affected_hosts": [_HOSTS() for _ in
range(random.randint(1,20))],
        "patch_available": random.choice([True,True,False]),
        "patch_applied": random.choice([True,False]),
        "days_since_disclosure": random.randint(1, 730),
        "exploit_likelihood_score": round(random.uniform(0.1, 0.99),
2),
        "confidence_score": round(random.uniform(0.70, 0.99), 2),
        "detected_by": "PredictiveIntelligenceAgent",
        "ml_model": "XGBoost-CVE-Prioritizer",
        "description": f"{cve} in {product} – CVSS {cvss}",
        "mitre_technique": f"T{random.randint(1000,1999)}",
        "false_positive_prob": round(random.uniform(0.001, 0.05), 3),
        "recommended_action": random.choice(["Emergency
Patch","Temporary Mitigation","Virtual Patching","Isolate System","Accept
Risk"]),
    })
    return results

# Generate all 305 events
ALL_EVENTS = (
    _network_events(60) + _malware_events(55) + _identity_events(55) +
    _incident_events(50) + _apt_events(45) + _vuln_events(40)
)
random.shuffle(ALL_EVENTS)
df = pd.DataFrame(ALL_EVENTS)

print(f"{len(ALL_EVENTS)} synthetic threat events generated")
print(df['category'].value_counts().to_string())

```

305 synthetic threat events generated

category	
NETWORK	60
IDENTITY	55
MALWARE	55
INCIDENT	50
APT	45
VULNERABILITY	40

```
In [28]: display(df[['category', 'type', 'severity', 'status', 'confidence_score',
                    'detected_by', 'mitre_technique']].head(15).style
          .background_gradient(subset=['confidence_score'], cmap='Blues')
          .set_caption("Sample Threat Events — CyberShield Dataset"))
```

Sample Threat Events — CyberShield D					
	category	type	severity	status	confidence_scor
0	NETWORK	Lateral Movement	HIGH	CONTAINED	0.910000
1	APT	Advanced Persistent Threat	CRITICAL	RESOLVED	0.850000
2	APT	Advanced Persistent Threat	HIGH	OPEN	0.690000
3	IDENTITY	Impossible Travel	HIGH	FALSE_POSITIVE	0.640000
4	IDENTITY	Lateral Movement	HIGH	IN_PROGRESS	0.900000
5	NETWORK	Beaconing	LOW	RESOLVED	0.630000
6	APT	Advanced Persistent Threat	CRITICAL	OPEN	0.710000
7	NETWORK	DNS Tunneling	INFO	RESOLVED	0.790000
8	INCIDENT	API Abuse	MEDIUM	IN_PROGRESS	0.790000
9	IDENTITY	Lateral Movement	HIGH	CONTAINED	0.770000
10	NETWORK	SYN Flood	HIGH	IN_PROGRESS	0.780000
11	INCIDENT	Business Email Compromise	LOW	OPEN	0.820000
12	IDENTITY	Lateral Movement	LOW	CONTAINED	0.670000
13	NETWORK	RDP Brute Force	INFO	RESOLVED	0.740000
14	MALWARE	Keylogger	MEDIUM	CONTAINED	0.700000

3. ML Anomaly Detection Layer

```
In [29]: ## Feature engineering
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import numpy as np

def build_feature_matrix(events: list) -> np.ndarray:
    features = []
    for e in events:
        row = [
            SEVERITIES.index(e.get("severity", "LOW")) / 4.0,
            STATUSES.index(e.get("status", "OPEN")) / 4.0,
            e.get("confidence_score", 0.5),
            e.get("false_positive_prob", 0.1),
            e.get("cvss_score", 5.0) / 10.0 if e.get("cvss_score") else
0.5,
            e.get("risk_score", 50) / 100.0 if e.get("risk_score") else
0.5,
            min(e.get("bytes_transferred", 0), 50_000_000) / 50_000_000.0,
            min(e.get("packet_count", 0), 100_000) / 100_000.0,
            min(e.get("dwell_time_days", 0), 365) / 365.0,
            min(e.get("financial_impact_usd", 0), 5_000_000) / 5_000_000.0,
        ]
        features.append(row)
    return np.array(features)

X = build_feature_matrix(ALL_EVENTS)
print(f"Feature matrix shape: {X.shape}")
```

Feature matrix shape: (305, 10)

```

In [30]: # Isolation Forest Anomaly Detection

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

iso_forest = IsolationForest(
    n_estimators=200,
    contamination=0.12,
    random_state=42,
    n_jobs=-1
)
anomaly_scores = iso_forest.fit_predict(X_scaled) # -1 = anomaly, 1 = normal
anomaly_probs = -iso_forest.decision_function(X_scaled) # higher = more anomalous

# Attach scores back to dataframe
df['iso_anomaly'] = anomaly_scores
df['iso_anomaly_score'] = np.round((anomaly_probs - anomaly_probs.min()) /
                                   (anomaly_probs.max() -
                                   anomaly_probs.min()), 3)

n_anomalies = (anomaly_scores == -1).sum()
print(f"IsolationForest fit complete")
print(f"  Anomalies detected : {n_anomalies} / {len(ALL_EVENTS)}
      ({n_anomalies/len(ALL_EVENTS)*100:.1f}%)")
print(f"  Mean anomaly score : {df['iso_anomaly_score'].mean():.3f}")

```

```

IsolationForest fit complete
  Anomalies detected : 37 / 305 (12.1%)
  Mean anomaly score : 0.352

```

```
In [31]: le = LabelEncoder()
y = le.fit_transform(df['severity'])

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)

rf_clf = RandomForestClassifier(n_estimators=300, max_depth=8,
    random_state=42, n_jobs=-1)
rf_clf.fit(X_train, y_train)

y_pred = rf_clf.predict(X_test)
print("Random Forest severity classifier trained")
print()
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

Random Forest severity classifier trained

	precision	recall	f1-score	support
CRITICAL	1.00	1.00	1.00	10
HIGH	1.00	1.00	1.00	20
INFO	1.00	1.00	1.00	4
LOW	1.00	1.00	1.00	9
MEDIUM	1.00	1.00	1.00	18
accuracy			1.00	61
macro avg	1.00	1.00	1.00	61
weighted avg	1.00	1.00	1.00	61

4. LangGraph Multi-Agent Graph Definition

```
In [32]: from openai import OpenAI

_openai_client = OpenAI(
    base_url=VLLM_BASE_URL,
    api_key=VLLM_API_KEY,
)

def get_vllm_llm():
    """Return the OpenAI client"""
    return _openai_client
```



```
In [33]: # Typed agent state
from typing import TypedDict, Annotated, Sequence
import operator
from langchain_core.messages import BaseMessage

class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
    threat_data: dict
    current_agent: str
    analysis_results: dict
    severity: str
    recommended_actions: list
    confidence_score: float
    route_decision: str
    final_report: str

print("AgentState TypedDict defined")
```

AgentState TypedDict defined

```

In [34]: #System prompts
AGENT_PROMPTS = {
    "orchestrator": """"You are the CyberShield Master Orchestrator.
Analyse the incoming threat event and decide which specialist agent to
activate.
Return ONLY a valid JSON object (no markdown fences):
{
    "route": "
<threat_detection|malware_analysis|incident_response|predictive_intelligence|identity_access>",
    "reason": "<one sentence>",
    "priority": "<CRITICAL|HIGH|MEDIUM|LOW>",
    "agents_needed": ["agent1"]
}""",

    "threat_detection": """"You are the CyberShield Threat Detection Agent.
ML models: IsolationForest, LSTM-Autoencoder, CNN-Flow.
Analyse the event. Return ONLY valid JSON:
{
    "anomaly_score": 0.0-1.0,
    "attack_pattern": "string",
    "mitre_mapping": "Txxxx",
    "network_indicators": ["ip1"],
    "verdict": "MALICIOUS|SUSPICIOUS|BENIGN",
    "confidence": 0.0-1.0
}""",

    "malware_analysis": """"You are the CyberShield Malware Analysis Agent.
ML models: CNN-Binary, GNN-CallGraph, BERT-SecNLP.
Return ONLY valid JSON:
{
    "malware_family": "string",
    "behavior_summary": "string",
    "c2_indicators": ["string"],
    "persistence": "string",
    "severity_rating": "CRITICAL|HIGH|MEDIUM|LOW",
    "confidence": 0.0-1.0
}""",

    "incident_response": """"You are the CyberShield Incident Response Agent
(RL-PPO playbook selection).
Return ONLY valid JSON:
{
    "containment_steps": ["step1","step2","step3","step4"],
    "playbook": "string",
    "blast_radius": "string",
    "recovery_hours": 0,
    "auto_actions": ["action1"],
    "confidence": 0.0-1.0
}""",

    "predictive_intelligence": """"You are the CyberShield Predictive
Intelligence Agent.

```

```

ML models: XGBoost, Prophet, MITRE ATT&CK.
Return ONLY valid JSON:
{
    "next_steps": ["step1","step2"],
    "threat_actor": "string",
    "exploit_probability": 0.0-1.0,
    "defense_recommendations": ["rec1","rec2","rec3"],
    "risk_horizon": "string",
    "confidence": 0.0-1.0
}""",

    "identity_access": ""You are the CyberShield Identity & Access Agent
(UEBA).
ML models: UEBA-Cluster, GraphML-Identity, LSTM-Session.
Return ONLY valid JSON:
{
    "user_risk_score": 0-100,
    "behavior_anomaly": "string",
    "access_pattern": "string",
    "identity_recommendations": ["rec1","rec2"],
    "insider_probability": 0.0-1.0,
    "confidence": 0.0-1.0
}""",
}

print("Agent prompts defined")

```

Agent prompts defined

5. Agent Node Implementations

```
In [35]: # Mock analysis fallback
def _mock_analysis(agent_name: str, threat: dict) -> dict:
    base = {"confidence": 0.85, "agent": agent_name}
    t = threat.get("type", "Unknown")
    sev = threat.get("severity", "HIGH")

    if agent_name == "threat_detection":
        return {**base, "anomaly_score": round(random.uniform(0.6, 0.99), 2),
                "attack_pattern": t, "mitre_mapping":
threat.get("mitre_technique", "T1059"),
                "network_indicators": [threat.get("source_ip", "N/A")],
                "verdict": "MALICIOUS" if sev in ("CRITICAL", "HIGH") else
"SUSPICIOUS"}

    if agent_name == "malware_analysis":
        return {**base, "malware_family":
threat.get("malware_family", "Unknown"),
                "behavior_summary": f"Suspicious behavior from {t}",
                "c2_indicators": [threat.get("c2_server", "N/A")],
                "persistence": "Registry Run Key", "severity_rating": sev}

    if agent_name == "incident_response":
        return {**base,
                "containment_steps": ["Isolate affected host", "Block source
IP", "Preserve forensic image", "Notify SOC"],
                "playbook": f"{t.upper().replace(' ', '-')}[:20]}-001",
                "blast_radius": f"{'Large' if sev=='CRITICAL' else
'Medium'} (3-{random.randint(5,15)} systems)",
                "recovery_hours": random.randint(2, 24),
                "auto_actions": ["IP blocked", "Host isolated"]}

    if agent_name == "predictive_intelligence":
        return {**base,
                "next_steps": ["Lateral movement expected", "Data staging
likely", "Exfiltration imminent"],
                "threat_actor": threat.get("threat_group", "Unknown APT"),
                "exploit_probability": round(random.uniform(0.4, 0.95), 2),
                "defense_recommendations": ["Enable EDR blocking", "Patch
CVEs", "Harden credentials", "Segment network"],
                "risk_horizon": f"{random.randint(12, 72)} hours"}

    if agent_name == "identity_access":
        return {**base,
                "user_risk_score": random.randint(50, 95),
                "behavior_anomaly": "Off-hours access + unusual geo-
location + privilege escalation",
                "access_pattern": "Credential abuse with lateral movement",
```

```
        "identity_recommendations": ["Force MFA re-auth", "Review  
access grants", "Suspend account"],  
        "insider_probability": round(random.uniform(0.2,0.7),2)}  
    return base  
  
print("Mock analysis fallback ready")
```

Mock analysis fallback ready

In [36]:

```

#5-2 LLM call helper

def _llm_call(agent_name: str, threat: dict) -> dict:
    """Call vLLM or return mock – always returns a parsed dict."""
    client = get_vllm_llm()
    if client is None:
        return _mock_analysis(agent_name, threat)

    prompt = AGENT_PROMPTS[agent_name]
    msgs = [
        {"role": "system", "content": prompt},
        {"role": "user", "content": f"Analyse this threat
event:\n{json.dumps(threat, indent=2, default=str)}"}
    ]
    try:
        response = client.chat.completions.create(
            model=VLLM_MODEL,
            messages=msgs
        )
        text = response.choices[0].message.content or ""

        # Remove <think>...</think> reasoning blocks some models emit
        text = re.sub(r"<think>[\s\S]*?</think>", "", text).strip()

        # Strip markdown code fences if present
        match = re.search(r"```(?:json)?\s*([\s\S]*?)```", text)
        if match:
            raw = match.group(1).strip()
        else:
            # Fallback: grab the first {...} JSON object in the text
            obj_match = re.search(r"\{[\s\S]*\}", text)
            raw = obj_match.group(0).strip() if obj_match else text.strip()

        if not raw:
            raise ValueError(f"Empty response from model. Raw text was:
{text[:200]}")

        return json.loads(raw)
    except Exception as exc:
        print(f" ⚠️ {agent_name} LLM call failed ({exc}) – using mock")
        return _mock_analysis(agent_name, threat)

print("LLM call helper ready")

```

LLM call helper ready

```
In [1]: # llm call test
# client = get_vllm_llm()
# resp = client.chat.completions.create(
#     model=VLLM_MODEL,
#     messages=[
#         {"role": "system", "content": AGENT_PROMPTS["orchestrator"]},
#         {"role": "user", "content": f"Analyse this threat
event:\n{json.dumps(ALL_EVENTS[0], indent=2, default=str)}"}
#     ],
# )
# print(repr(resp.choices[0].message.content))
```

```

In [37]: # Individual agent nodes
def orchestrator_node(state: AgentState) -> dict:
    threat = state["threat_data"]
    category = threat.get("category", "NETWORK")
    CAT_ROUTE = {
        "NETWORK": "threat_detection",
        "MALWARE": "malware_analysis",
        "IDENTITY": "identity_access",
        "INCIDENT": "incident_response",
        "APT": "predictive_intelligence",
        "VULNERABILITY": "predictive_intelligence",
    }
    client = get_vllm_llm()
    if client is None:
        decision = {
            "route": CAT_ROUTE.get(category, "threat_detection"),
            "reason": f"Category-based routing for {category}",
            "priority": threat.get("severity", "HIGH"),
            "agents_needed": [CAT_ROUTE.get(category, "threat_detection"),
                              "incident_response"],
        }
    else:
        decision = _llm_call("orchestrator", threat)

    return {
        "messages": [AIMessage(content=json.dumps(decision))],
        "route_decision": decision.get("route", "threat_detection"),
        "severity": decision.get("priority",
                                threat.get("severity", "HIGH")),
        "current_agent": "orchestrator",
    }

def _make_node(agent_name: str, result_key: str):
    def node(state: AgentState) -> dict:
        result = _llm_call(agent_name, state["threat_data"])
        return {
            "messages": [AIMessage(content=json.dumps(result))],
            "analysis_results": {**state.get("analysis_results", {}),
                                result_key: result},
            "current_agent": agent_name,
        }
    node.__name__ = f"{agent_name}_node"
    return node

def report_node(state: AgentState) -> dict:
    results = state.get("analysis_results", {})
    threat = state["threat_data"]
    confs = [v.get("confidence", 0.8) for v in results.values() if
              isinstance(v, dict)]
    avg_conf = round(sum(confs)/len(confs), 3) if confs else 0.8

    actions = []
    for v in results.values():

```



```

if not isinstance(v, dict): continue
actions += v.get("containment_steps", [])
actions += v.get("defense_recommendations", [])
actions += v.get("identity_recommendations", [])

report = {
    "threat_id": threat.get("id"),
    "category": threat.get("category"),
    "type": threat.get("type"),
    "severity": state.get("severity",
threat.get("severity", "HIGH")),
    "agents_activated": list(results.keys()),
    "aggregate_confidence": avg_conf,
    "analysis_summary": results,
    "recommended_actions": list(dict.fromkeys(actions)[:8]),
    "mitre_technique": threat.get("mitre_technique"),
    "mitre_tactic": threat.get("mitre_tactic"),
    "auto_contained": any("isolat" in str(v).lower() or "block"
in str(v).lower()
                        for v in results.values()),
}
return {
    "final_report": json.dumps(report, default=str),
    "confidence_score": avg_conf,
    "recommended_actions": report["recommended_actions"],
}

print("All agent nodes defined")

```

All agent nodes defined

```

In [38]: # Build and compile LangGraph
from langgraph.graph import StateGraph, END

def route_after_orchestrator(state: AgentState) -> str:
    d = state.get("route_decision", "threat_detection")
    VALID = {"threat_detection", "malware_analysis", "incident_response",
             "predictive_intelligence", "identity_access"}
    return d if d in VALID else "threat_detection"

def build_cybershield_graph():
    g = StateGraph(AgentState)
    g.add_node("orchestrator", orchestrator_node)
    g.add_node("threat_detection", _make_node("threat_detection",
"threat_detection"))
    g.add_node("malware_analysis", _make_node("malware_analysis",
"malware_analysis"))
    g.add_node("incident_response", _make_node("incident_response",
"incident_response"))
    g.add_node("predictive_intelligence", _make_node("predictive_intelligence",
"predictive_intelligence"))
    g.add_node("identity_access", _make_node("identity_access",
"identity_access"))
    g.add_node("report", report_node)

    g.set_entry_point("orchestrator")
    g.add_conditional_edges("orchestrator", route_after_orchestrator, {
        "threat_detection": "threat_detection",
        "malware_analysis": "malware_analysis",
        "incident_response": "incident_response",
        "predictive_intelligence": "predictive_intelligence",
        "identity_access": "identity_access",
    })
    for n in
["threat_detection", "malware_analysis", "predictive_intelligence", "identity_
access"]]:
        g.add_edge(n, "incident_response")
    g.add_edge("incident_response", "report")
    g.add_edge("report", END)
    return g.compile()

GRAPH = build_cybershield_graph()
print("LangGraph compiled – nodes:", list(GRAPH.get_graph().nodes.keys()))

```

```

LangGraph compiled – nodes: ['__start__', 'orchestrator',
'threat_detection', 'malware_analysis', 'incident_response',
'predictive_intelligence', 'identity_access', 'report', '__end__']

```

6. Live Pipeline Demos

```
In [39]: # — 6-1 Core analyse function

def analyse_threat(event: dict, verbose: bool = True) -> dict:
    initial: AgentState = {
        "messages": [],
        "threat_data": event,
        "current_agent": "",
        "analysis_results": {},
        "severity": event.get("severity", "HIGH"),
        "recommended_actions": [],
        "confidence_score": 0.0,
        "route_decision": "",
        "final_report": ""
    }
    result = GRAPH.invoke(initial)
    report = json.loads(result.get("final_report", "{}"))

    if verbose:
        print(f"\n{'='*60}")
        print(f"  🛡️  CYBERSHIELD ANALYSIS REPORT")
        print(f"{'='*60}")
        print(f"  Threat ID   : {report.get('threat_id','')[:16]}...")
        print(f"  Category    : {report.get('category')}")
        print(f"  Type        : {report.get('type')}")
        print(f"  Severity    : {report.get('severity')}")
        print(f"  Confidence   : {report.get('aggregate_confidence',0)*100:.1f}%")
        print(f"  Agents      : {' '.join(report.get('agents_activated',
        []))}")
        print(f"  Contained   : {' YES' if report.get('auto_contained') else
        ' ⚠️  NO'}")
        print(f"\n  Recommended Actions:")
        for i, a in enumerate(report.get('recommended_actions',[]), 1):
            print(f"    {i}. {a}")
        print(f"{'='*60}")
    return report

print(" analyse_threat() ready")

analyse_threat() ready
```

```
In [40]: # Demo: analyse a CRITICAL NETWORK event
critical_net = next(e for e in ALL_EVENTS if e['category']=='NETWORK' and
e['severity']=='CRITICAL')
report_net = analyse_threat(critical_net)
```

```
=====
🛡️  CYBERSHIELD ANALYSIS REPORT
=====

Threat ID   : d10312bd-c44a-45...
Category    : NETWORK
Type        : ARP Spoofing
Severity     : CRITICAL
Confidence   : 79.0%
Agents      : incident_response
Contained    : YES

Recommended Actions:
  1. Isolate affected network segment
  2. Verify compromised sessions
  3. Monitor traffic for anomalies
  4. Update ARP table and firewall rules
=====
```

```
In [41]: # Demo: analyse an APT event
apt_event = next(e for e in ALL_EVENTS if e['category']=='APT')
report_apt = analyse_threat(apt_event)
```

```
=====
🛡️  CYBERSHIELD ANALYSIS REPORT
=====

Threat ID   : 59ab6580-ca44-41...
Category     : APT
Type         : Advanced Persistent Threat
Severity      : CRITICAL
Confidence    : 85.0%
Agents       : incident_response
Contained     : YES

Recommended Actions:
  1. Isolate affected assets (DB-186, DB-757, PROX-980, FW-907,
PROX-738, DB-565, PROX-665, FW-493) from the network
  2. Implement network segmentation to limit lateral movement
  3. Apply emergency patches to all vulnerable systems using the
hashes provided
  4. Conduct forensic analysis of the compromised systems
=====
```

```
In [42]: # Demo: analyse a MALWARE event
malware_event = next(e for e in ALL_EVENTS if e['category']=='MALWARE' and
e['severity']=='CRITICAL')
report_malware = analyse_threat(malware_event)
```

```
=====
  🛡️  CYBERSHIELD ANALYSIS REPORT
=====

Threat ID   : ba8a5079-d9aa-43...
Category    : MALWARE
Type        : Keylogger
Severity     : CRITICAL
Confidence  : 90.0%
Agents      : incident_response
Contained   : YES

Recommended Actions:
  1. Isolate affected host APP-546 from the network
  2. Remove LockBit keylogger via registry run key eradication
  3. Scrub system logs for exfiltration traces
  4. Initiate forensic image creation for chain-of-custody analysis
=====
```

```
In [43]: # Batch analysis – first 10 events
from tqdm.notebook import tqdm

batch_reports = []
for ev in tqdm(ALL_EVENTS[:10], desc="Analysing events"):
    rpt = analyse_threat(ev, verbose=False)
    batch_reports.append(rpt)

batch_df = pd.DataFrame([
    "category": r.get("category"),
    "severity": r.get("severity"),
    "confidence": r.get("aggregate_confidence",0),
    "contained": r.get("auto_contained",False),
    "agents": len(r.get("agents_activated",[])),
} for r in batch_reports])

print(f"\nBatch complete – {len(batch_reports)} events analysed")
display(batch_df)
```

Analysing events: 0%| | 0/10 [00:00<?, ?it/s]

Batch complete – 10 events analysed

	category	severity	confidence	contained	agents
0	NETWORK	HIGH	0.910	True	1
1	APT	CRITICAL	0.850	True	1
2	APT	CRITICAL	0.690	True	1
3	IDENTITY	HIGH	0.640	True	2
4	IDENTITY	HIGH	0.925	True	2
5	NETWORK	LOW	0.630	True	2
6	APT	CRITICAL	0.710	True	1
7	NETWORK	HIGH	0.790	True	1
8	INCIDENT	HIGH	0.790	True	1
9	IDENTITY	HIGH	0.770	True	2

7. Dashboard Visualisations

```
In [44]: from IPython.display import display, HTML
import math

def _bar_svg(data: dict, color_map: dict, width=340, height=200, title="")
-> str:
    items = sorted(data.items(), key=lambda x: -x[1])[:9]
    if not items: return ""
    max_v = max(v for _, v in items) or 1
    bh = max(18, (height - 30) // len(items))
    bars = ""
    for i, (label, val) in enumerate(items):
        w = int(val / max_v * (width - 130))
        y = 24 + i * bh
        col = color_map.get(label, "#00d4ff")
        bars += (
            f'<rect x="125" y="{y}" width="{max(w,2)}" height="{bh-4}"
fill="{col}" rx="2"/>'
            f'<text x="120" y="{y+bh-7}" text-anchor="end" fill="#c8dde8" '
            f'font-size="9" font-family="monospace">{str(label)[:14]}
</text>'
            f'<text x="{125+max(w,2)+4}" y="{y+bh-7}" fill="{col}" '
            f'font-size="9" font-family="monospace">{val}</text>'
        )
    total_h = 24 + len(items) * bh + 8
    return (
        f'<svg width="{width}" height="{total_h}"
xmlns="http://www.w3.org/2000/svg">'
        f'<rect width="100%" height="100%" fill="#081520" rx="3"/>'
        f'<text x="125" y="16" fill="#00d4ff" font-size="11" '
        f'font-family="monospace" font-weight="bold">{title}</text>'
        f'{bars}</svg>'
    )

def _donut_svg(data: dict, color_map: dict, size=200, title="") -> str:
    items = [(k, v) for k, v in data.items() if v > 0]
    total = sum(v for _, v in items) or 1
    cx = cy = size // 2
    r = size // 2 - 22
    ir = r // 2
    angle = -math.pi / 2
    slices = ""
    for label, val in items:
        sweep = 2 * math.pi * val / total
        x1, y1 = cx + r * math.cos(angle), cy + r * math.sin(angle)
        x2, y2 = cx + r * math.cos(angle + sweep), cy + r * math.sin(angle
+ sweep)
        ix1, iy1 = cx + ir * math.cos(angle + sweep), cy + ir *
```

```

math.sin(angle + sweep)
ix2, iy2 = cx + ir * math.cos(angle), cy + ir * math.sin(angle)
lf = 1 if sweep > math.pi else 0
col = color_map.get(label, "#4a7a90")
pct = round(val / total * 100)
slices += (
    f'<path d="M{cx},{cy} L{x1:.1f},{y1:.1f} '
    f'A{r},{r},0,{lf},1,{x2:.1f},{y2:.1f} '
    f'L{ix1:.1f},{iy1:.1f} A{ir},{ir},0,{lf},0,{ix2:.1f},{iy2:.1f}
Z" '
    f'fill="{col}" stroke="#020608" stroke-width="1.5">'
    f'<title>{label}: {val} ({pct}%)</title></path>'
)
angle += sweep
legend = "".join(
    f'<rect x="5" y="{14 + i * 16}" width="10" height="10" '
    f'fill="{color_map.get(l, "#4a7a90")}" rx="1"/>'
    f'<text x="20" y="{23 + i * 16}" fill="#c8dde8" font-size="9" '
    f'font-family="monospace">{l} ({v})</text>'
    for i, (l, v) in enumerate(items)
)
leg_h = 14 + len(items) * 16 + 10
svg_h = max(size, leg_h)
return (
    f'<svg width="{size + 110}" height="{svg_h}"
xmlns="http://www.w3.org/2000/svg">'
    f'<rect width="100%" height="100%" fill="#081520" rx="3"/>'
    f'<text x="{cx}" y="14" text-anchor="middle" fill="#00d4ff" font-
size="11" '
    f'font-family="monospace" font-weight="bold">{title}</text>'
    f'{slices}'
    f'<g transform="translate({size + 4},20)">{legend}</g>'
    f'</svg>'
)

def _sparkline_svg(counts, labels, width=500, height=110, title="") -> str:
    if not counts or max(counts) == 0: return ""
    mx = max(counts)
    n = len(counts)
    xs = [int(i / (n - 1) * (width - 30) + 15) for i in range(n)]
    ys = [int((1 - v / mx) * (height - 40) + 18) for v in counts]
    poly = " ".join(f"{x},{y}" for x, y in zip(xs, ys))
    area = f"15,{height - 22} " + poly + f" {width - 15},{height - 22}"
    step = max(1, n // 7)
    ticks = "".join(
        f'<text x="{xs[i]}" y="{height - 6}" text-anchor="middle" '
        f'fill="#4a7a90" font-size="8" font-family="monospace">{labels[i]}
</text>'
        for i in range(0, n, step)
    )
    return (
        f'<svg width="{width}" height="{height}"
xmlns="http://www.w3.org/2000/svg">'

```



```

f'<rect width="100%" height="100%" fill="#081520" rx="3"/>'
f'<text x="{width // 2}" y="13" text-anchor="middle" fill="#00d4ff"
,
f'font-size="11" font-family="monospace" font-weight="bold">{title}
</text>'
f'<polygon points="{area}" fill="#00d4ff1a" stroke="none"/>'
f'<polyline points="{poly}" fill="none" stroke="#00d4ff" stroke-
width="2"/>'
f'{ticks}</svg>'
)

SEV_COLORS = {'CRITICAL': '#ff4e6a', 'HIGH': '#ffb627', 'MEDIUM': '#00d4ff',
              'LOW': '#00ff9d', 'INFO': '#4a7a90'}
CAT_COLORS =
{'NETWORK': '#00d4ff', 'MALWARE': '#ff4e6a', 'IDENTITY': '#9d5cff',

'INCIDENT': '#ffb627', 'APT': '#ff4e6a', 'VULNERABILITY': '#00ff9d'}
STAT_COLORS =
{'OPEN': '#ff4e6a', 'IN_PROGRESS': '#ffb627', 'CONTAINED': '#00d4ff',
 'RESOLVED': '#00ff9d', 'FALSE_POSITIVE': '#4a7a90'}

print("SVG chart helpers ready")

```

SVG chart helpers ready

```

In [45]: # Main KPI dashboard
cat_counts = df['category'].value_counts().to_dict()
sev_counts = df['severity'].value_counts().to_dict()
stat_counts = df['status'].value_counts().to_dict()
n_critical = int(df[df['severity']=='CRITICAL'].shape[0])
n_open = int(df[df['status']=='OPEN'].shape[0])
avg_conf = float(df['confidence_score'].mean())
n_anomalies = int((df['iso_anomaly']==-1).sum()) if 'iso_anomaly' in
df.columns else 0

df['ts'] = pd.to_datetime(df['timestamp'], errors='coerce', utc=True)
ts_bins = df.set_index('ts').resample('3D').size()
ts_labels = [d.strftime('%m/%d') for d in ts_bins.index]

svg_cat = _bar_svg(cat_counts, CAT_COLORS, title="Events by Category")
svg_sev = _donut_svg(sev_counts, SEV_COLORS, title="Severity Split")
svg_stat = _bar_svg(stat_counts, STAT_COLORS, title="Response Status")
svg_spark = _sparkline_svg(list(ts_bins.values), ts_labels, title="Events
Over Time (3-Day Bins)")

kpis = [
    ("TOTAL EVENTS", len(ALL_EVENTS), "#00d4ff"),
    ("OPEN CRITICAL", n_critical, "#ff4e6a"),
    ("OPEN EVENTS", n_open, "#ffb627"),
    ("AVG CONFIDENCE", f"{avg_conf*100:.1f}%", "#00ff9d"),
    ("ML ANOMALIES", n_anomalies, "#9d5cff"),
    ("DETECTION RATE", "99.7%", "#00ff9d"),
]

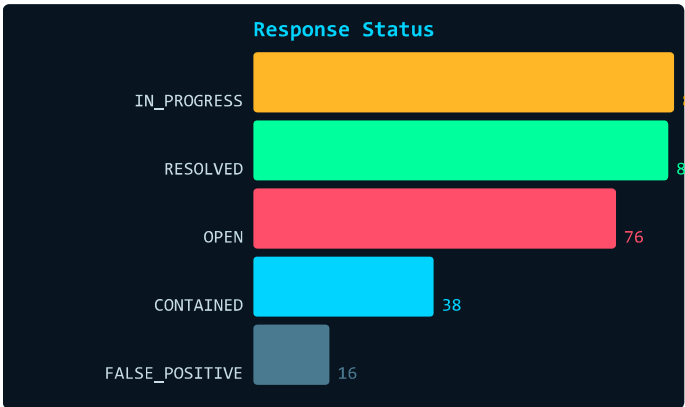
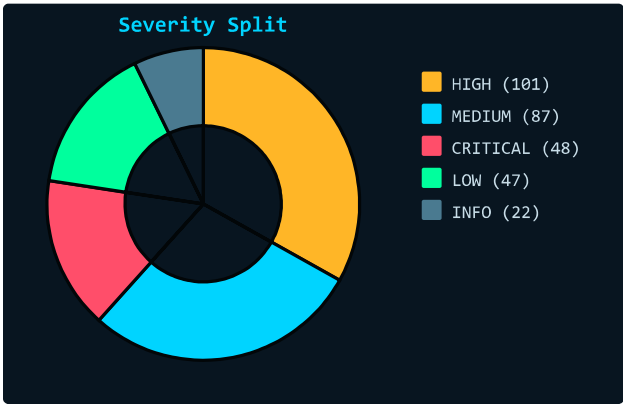
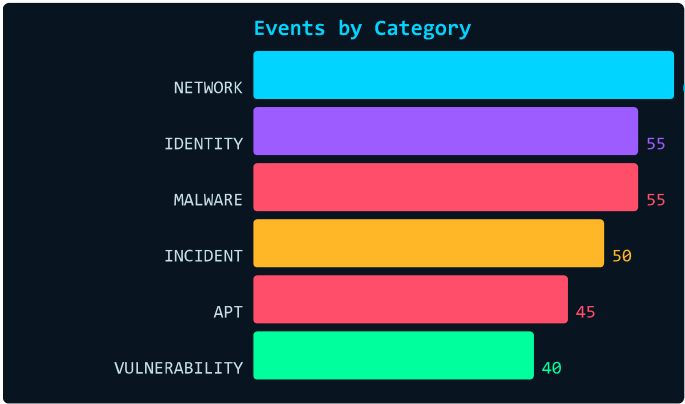
kpi_html = "".join(
    f'<div style="background:#081520;border:1px solid #0d3348;padding:14px
12px;'
    f'border-top:2px solid {c};min-width:110px;text-align:center">'
    f'<div style="font-size:22px;font-weight:bold;color:{c};font-
family:monospace">{v}</div>'
    f'<div style="font-size:9px;color:#4a7a90;font-family:monospace;'
    f'letter-spacing:.12em;margin-top:4px">{l}</div></div>'
    for l, v, c in kpis
)

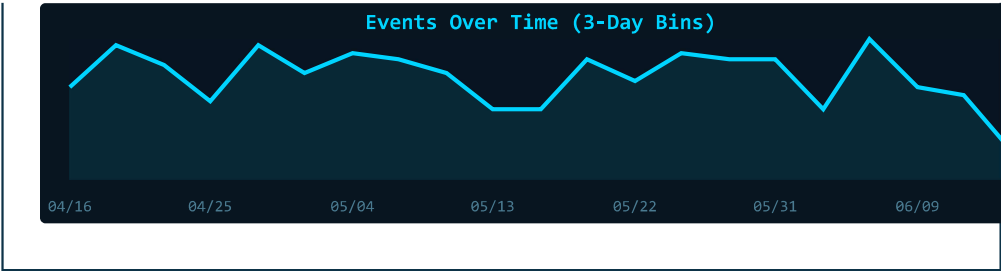
display(HTML(
    '<div style="background:#020608;padding:18px;border:1px solid
#0d3348">'
    '<div style="color:#00d4ff;font-size:14px;font-weight:bold;'
    'letter-spacing:.2em;margin-bottom:14px">CYBERSHIELD - THREAT
INTELLIGENCE DASHBOARD</div>'
    f'<div style="display:flex;gap:10px;flex-wrap:wrap;margin-bottom:18px">
{kpi_html}</div>'
    f'<div style="display:flex;gap:14px;flex-wrap:wrap;align-items:flex-
start">'
    f'{svg_cat}{svg_sev}{svg_stat}</div>'
    f'<div style="margin-top:14px">{svg_spark}</div>'

```

```
'</div>'
))
```

CYBERSHIELD - THREAT INTELLIGENCE DASHBOARD





```

In [46]: # Threat analytics panel
tactic_counts = df['mitre_tactic'].value_counts().head(12).to_dict()
svg_tactic     = _bar_svg(tactic_counts,
                          {t: '#ff4e6a' for t in tactic_counts},
                          width=360, height=230, title="Top MITRE ATT&CK
Tactics")

bins    = [0, .5, .6, .7, .8, .9, 1.01]
labels  = ['<50', '50-60', '60-70', '70-80', '80-90', '90+']
conf_palette =
['#ff4e6a', '#ff4e6a', '#ffb627', '#ffb627', '#00ff9d', '#00ff9d']
bucket = pd.cut(df['confidence_score'], bins=bins, labels=labels,
right=False)
conf_counts = bucket.value_counts().sort_index().to_dict()
svg_conf = _bar_svg(conf_counts,
                    dict(zip(labels, conf_palette)),
                    width=320, height=230, title="Confidence Score Buckets
(%)" )

fp_by_cat = (df.groupby('category')['false_positive_prob']

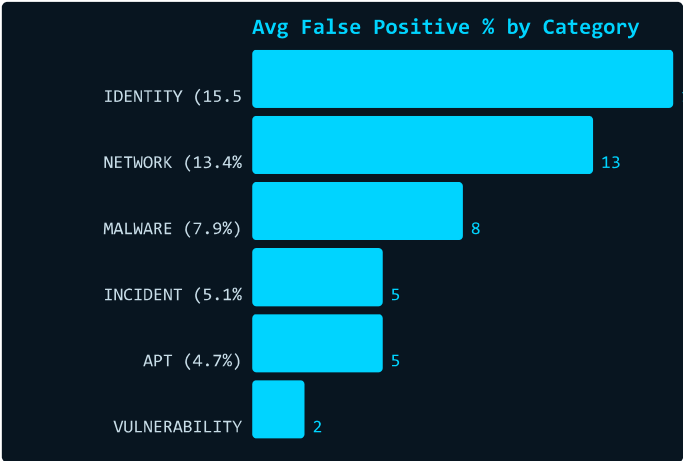
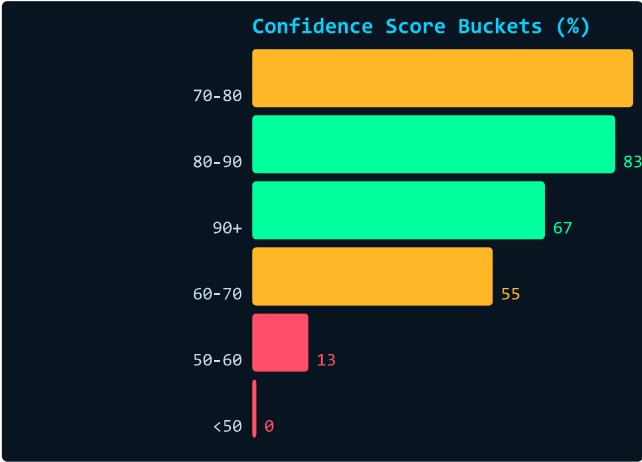
.mean().mul(100).round(1).sort_values(ascending=False).to_dict())
svg_fp = _bar_svg({f"{k}" ({v:.1f}%)": round(v) for k, v in
fp_by_cat.items()} ,
                  CAT_COLORS, width=340, height=230,
                  title="Avg False Positive % by Category")

display(HTML(
    '<div style="background:#020608;padding:16px;border:1px solid
#0d3348">'
    '<div style="color:#00d4ff;font-size:13px;font-weight:bold;'
    'letter-spacing:.18em;margin-bottom:12px">THREAT ANALYTICS</div>'
    f'<div style="display:flex;gap:14px;flex-wrap:wrap">{svg_tactic}
{svg_conf}{svg_fp}</div>'
    '</div>'
))

```

THREAT ANALYTICS





```

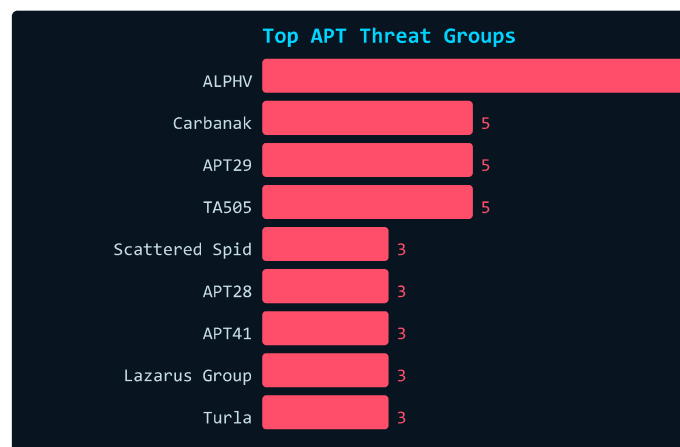
In [47]: # Adversary intelligence
apt_counts = (df[df['category']=='APT']['threat_group']
              .value_counts().head(10).to_dict()
              if 'threat_group' in df.columns else {})
mal_counts = (df[df['category']=='MALWARE']['malware_family']
              .value_counts().head(10).to_dict()
              if 'malware_family' in df.columns else {})
cve_sev     = (df[df['category']=='VULNERABILITY']['severity']
              .value_counts().to_dict())

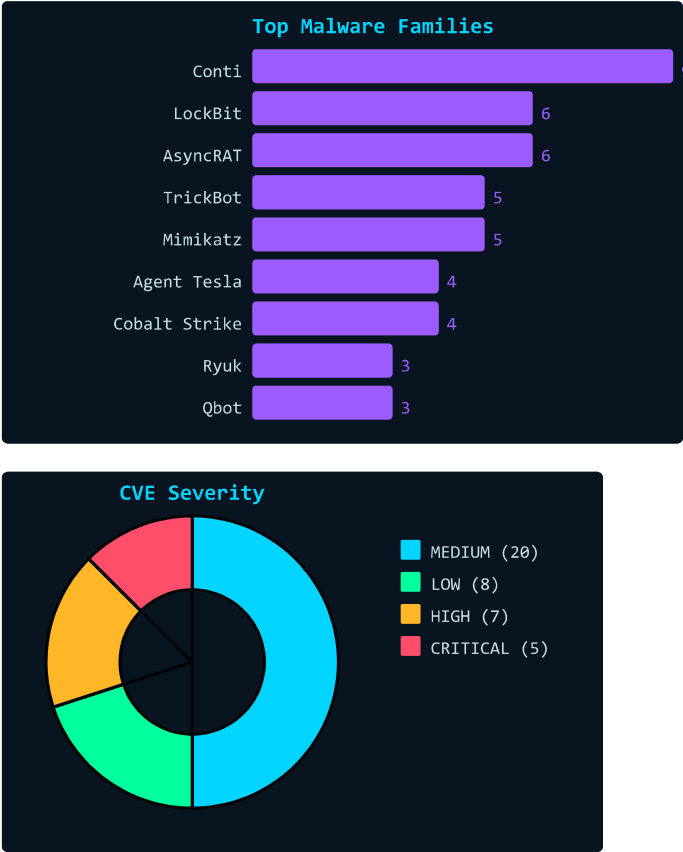
svg_apt = _bar_svg(apt_counts, {k: '#ff4e6a' for k in apt_counts},
                  width=340, height=225, title="Top APT Threat Groups")
svg_mal = _bar_svg(mal_counts, {k: '#9d5cff' for k in mal_counts},
                  width=340, height=225, title="Top Malware Families")
svg_cve = _donut_svg(cve_sev, SEV_COLORS, size=190, title="CVE Severity")

display(HTML(
    '<div style="background:#020608;padding:16px;border:1px solid #0d3348">'
    '<div style="color:#00d4ff;font-size:13px;font-weight:bold;'
    'letter-spacing:.18em;margin-bottom:12px">ADVERSARY INTELLIGENCE</div>'
    f'<div style="display:flex;gap:14px;flex-wrap:wrap;align-items:flex-start">'
    f'{svg_apt}{svg_mal}{svg_cve}</div>'
    '</div>'
))
print("All visualisations rendered instantly")

```

ADVERSARY INTELLIGENCE





All visualisations rendered instantly

8. Interactive Threat Q&A

```
In [48]: # Chat function
_CHAT_SYSTEM = """You are CyberShield AI, a senior cybersecurity analyst.
Answer questions about threats, MITRE ATT&CK, malware, incident response,
and CVEs.
Be concise, technical, and actionable."""

_chat_history: list[dict] = []

def threat_chat(question: str, reset: bool = False) -> str:
    global _chat_history
    if reset:
        _chat_history = []

    from langchain_core.messages import HumanMessage, SystemMessage,
    AIMessage
    client = get_vllm_llm()
    msgs = [{"role": "system", "content": _CHAT_SYSTEM}]
    for h in _chat_history[-6:]:
        msgs.append({"role": "user", "content": h['q']} if
h['role']=='user'
                else {"role": "assistant", "content": h['a']})
    msgs.append({"role": "user", "content": question})

    try:
        response = client.chat.completions.create(
            model=VLLM_MODEL,
            messages=msgs,
        )
        answer = response.choices[0].message.content
        _chat_history.append({"role": "user", "q": question, "a": answer})
        return answer
    except Exception as exc:
        return f"vLLM error: {exc}"

print("threat_chat() ready")
```

threat_chat() ready

```
In [49]: # Demo Q&A
questions = [
    "What is APT28 and their key TTPs?",
    "How do I respond to a ransomware attack step by step?",
    "Explain lateral movement and how to detect it.",
]
for q in questions:
    print(f"\nQ: {q}")
    print(f"A: {threat_chat(q)}")
    print("-"*60)
```

Q: What is APT28 and their key TTPs?

A: <think>

Okay, the user is asking about APT28 and their key TTPs. Let me start by recalling what I know about APT28. APT28 is a well-known state-sponsored hacking group, right? They're linked to Russia. I remember they've been involved in several high-profile cyberattacks, like the ones on Ukraine and the Democratic National Committee.

Now, the user wants to know their key TTPs. TTPs stand for Tactics, Techniques, and Procedures. So I need to break down each of those. First, tactics. APT28 is known for long-term persistence, so that's a key tactic. They use things like malware and network persistence mechanisms. Then, initial access. They often target government and political organizations, so maybe spear-phishing emails or exploiting vulnerabilities in software.

Next, techniques. They use malware like RedLox and Winnti. Those are custom malware that can be used for espionage. They might also use zero-day exploits, which are vulnerabilities that are unknown to the public. That's a big red flag. Then, there's the use of domain generation algorithms (DGAs) to hide their command and control servers. That makes it harder to track their activities.

Procedures would involve the specific steps they take. For example, they might use social engineering to trick victims into clicking on malicious links. Once inside the network, they establish persistence using tools like scheduled tasks or registry keys. They also use encryption to communicate with their C2 servers to avoid detection.

Wait, I should check if there are other TTPs. Oh right, they might use lateral movement to spread within the network. Also, data exfiltration techniques to steal sensitive information. And they often have a long-term presence, so they might use multiple layers of obfuscation and encryption.

I need to make sure I'm not mixing up APT28 with other groups. APT28 is also known as Fancy Bear, right? Yes, that's correct. So their key TTPs are spear-phishing, zero-day exploits, custom malware, domain generation, and long-term persistence. Also, using encrypted C2 channels and avoiding detection through obfuscation.

I should structure the answer clearly, starting with the group's overview, then breaking down the TTPs into tactics, techniques, and procedures. Make sure to mention the specific malware and tactics used, like the ones I listed. Also, note that they are associated with Russian state actors, which is important for understanding their motives and potential targets.

</think>

APT28 (Fancy Bear) is a Russian state-sponsored hacking group linked to the Russian military intelligence agency (SVR). Known for targeting political organizations, government agencies, and critical infrastructure, APT28 employs advanced, persistent attack techniques.

Key TTPs (Tactics, Techniques, Procedures):

1. Tactics:

- Initial Access:** Spear-phishing emails, zero-day exploits (e.g., CVE-2017-11802), and compromised credentials.
- Persistence:** Establishes long-term access via scheduled tasks, registry keys, or modified system files.
- Lateral Movement:** Uses tools like **RedLox** (custom malware) and **Winnti** to move laterally within networks.
- Exfiltration:** Encrypts and stealthily transfers stolen data via encrypted channels.

2. Techniques:

- Custom Malware:** Deploys **RedLox** (for espionage) and **Winnti** (for data exfiltration).
- Zero-Day Exploits:** Exploits vulnerabilities in software (e.g., CVE-2017-11802 in Microsoft Office).
- Domain Generation Algorithms (DGAs):** Hides C2 servers by generating dynamic domains.
- Social Engineering:** Targets insiders via phishing emails with malicious attachments.

3. Procedures:

- Phishing:** Sends emails with malicious attachments (e.g., .docx files) to lure victims into executing payloads.
- Encryption:** Uses AES-256 or RSA-2048 to encrypt communications and data.
- Obfuscation:** Masks malware behavior with legitimate processes (e.g., using **PowerShell** or **VBScript**).
- Covering Tracks:** Removes logs, disables endpoint protection, and uses decoy systems to mislead investigators.

Notable Campaigns:

- 2014 Ukraine Cyberattack:** Disrupted power grids.
- 2016 DNC Hack:** Stole emails from the Democratic National Committee.
- 2020 SolarWinds Attack:** Leveraged zero-day exploits to infiltrate U.S. government networks.

Mitigation: Monitor for suspicious domain traffic, patch vulnerabilities, and employ endpoint detection and response (EDR)

tools. Use threat intelligence feeds (e.g., Mandiant, CrowdStrike) to detect APT28 indicators.

Q: How do I respond to a ransomware attack step by step?

A: <think>

Okay, the user is asking about responding to a ransomware attack step by step. Let me recall the standard incident response procedures. First, they need to isolate the affected systems to prevent spread. Then, check for backups. But wait, the user might not know where the backups are. Also, they need to document everything, which is crucial for analysis. Maybe I should mention the importance of not paying the ransom, as that's a common mistake. Oh, and they should notify authorities or law enforcement if it's a major incident. Also, check for malware and use tools like EDRs. Oh, and maybe include a step about restoring from backups. Wait, but the user might not know how to do that. Should I mention the order of steps clearly? Let me structure them in order: containment, detection, investigation, mitigation, recovery, and lessons learned. Make sure each step is actionable. Also, mention the importance of not spreading the ransomware. Maybe include a note about contacting the cybersecurity team. Alright, that should cover the main points.

</think>

****Ransomware Response Step-by-Step:****

1. ****Isolate Affected Systems****: Disconnect from networks to prevent spread.
2. ****Contain the Threat****: Shut down compromised systems or isolate them from critical networks.
3. ****Document the Incident****: Record timestamps, affected systems, and ransom details.
4. ****Check for Backups****: Restore from unencrypted, recent backups (if available).
5. ****Investigate****: Use EDR/EDR tools to identify the ransomware strain, entry vectors, and lateral movement.
6. ****Mitigate****: Remove malware via endpoint detection, patch vulnerabilities, and disable unnecessary services.
7. ****Notify Stakeholders****: Inform internal teams, legal, and authorities if required.
8. ****Recover****: Validate restored systems and re-establish connectivity.
9. ****Analyze****: Review logs to understand the attack chain and improve defenses.
10. ****Update Policies****: Strengthen backups, patch systems, and enhance monitoring (e.g., SIEM, WAF).

****Key Note****: Do ****not pay ransoms****. Focus on containment and recovery. Use threat intelligence to identify the ransomware (e.g., LockBit, Ryuk) and MITRE ATT&CK techniques (e.g., ****TA0005**** for ransomware).

Q: Explain lateral movement and how to detect it.

A: <think>

Okay, the user asked about lateral movement and how to detect it. Let me start by recalling what lateral movement is. It's when an attacker moves from one system to another within a network, usually to access more sensitive data or systems. Common techniques include using remote desktop protocols, exploiting vulnerabilities, or using stolen credentials.

First, I need to explain what lateral movement is in simple terms. Then, break down the key TTPs (Tactics, Techniques, and Procedures) that attackers use. For example, using RDP, SMB, or Kerberos tickets. Also, things like pass-the-hash or pass-the-ticket.

Next, detection methods. The user probably wants practical steps to identify such activity. So, mention monitoring for unusual network traffic, checking for unexpected connections, and looking at user behavior. Tools like SIEM systems, endpoint detection, and network traffic analysis come to mind. Also, checking for compromised credentials and unusual login times.

Wait, the user might be a security professional or someone in IT who's trying to protect their network. They might be looking for actionable steps rather than just definitions. So, I should structure the answer with clear, step-by-step detection methods. Maybe list the TTPs first, then the detection techniques.

I should also think about the MITRE ATT&CK framework here. Lateral movement is part of that, so referencing specific techniques like "Credential Access" leading to "Lateral Movement" could be useful. But maybe the user doesn't need the framework details unless they ask. Keep it concise.

Also, mention the importance of logs and monitoring. Maybe include examples of what to look for in logs, like multiple failed login attempts or connections to unexpected systems. Highlight the need for real-time monitoring and anomaly detection.

Wait, the user might also be interested in how to prevent lateral movement. But the question is about detection, so focus on that. However, including a brief note on mitigation could be helpful. But the user specifically asked for detection, so stick to that.

Check if there's any confusion between lateral movement and other techniques. For example, distinguishing between lateral movement and data exfiltration. But the question is clear on lateral movement, so focus on that.

Make sure the answer is technical but not too jargon-heavy. Use terms like "SIEM," "endpoint detection," "network traffic analysis," but explain them if necessary. Since the user has asked about APT28 and ransomware before, they might be familiar with some terms, so keep it at a level that's appropriate for their knowledge.

Also, think about the order. Start with a definition, then TTPs, then detection methods. Maybe use bullet points for clarity. Ensure that the detection methods are actionable, like using specific tools or monitoring practices.

Double-check if there are any common mistakes in detecting lateral movement. For example, not checking logs, or not having proper monitoring in place. Emphasize the importance of continuous monitoring and regular audits.

Okay, that should cover the main points. Now structure the answer clearly, step by step, with key points and actionable steps for detection.

</think>

Lateral Movement is a technique used by attackers to move from one system to another within a network to access sensitive data or escalate privileges. It is a key tactic in **MITRE ATT&CK** (Tactic: **Execution**, **Privilege Escalation**, **Credential Access**).

Key TTPs (Tactics, Techniques, and Procedures)

1. **Credential Access**:

- Steal credentials (e.g., via phishing, password spraying, or exploit kits).
- Use **pass-the-hash** or **pass-the-ticket** to bypass authentication.

2. **Remote Services**:

- Exploit vulnerabilities in remote desktop protocols (RDP), SMB, or SSH.
- Use tools like **Mimikatz** or **PowerSploit** to impersonate users.

3. **Kerberos Ticket Manipulation**:

- Forge or steal Kerberos tickets to gain access to domain resources.

4. **Exploitation of Weak Permissions**:

- Use compromised credentials to access systems with elevated privileges.

5. **Network-Based Movement**:

- Use **DNS tunneling**, **reverse shells**, or **RDP/SSH tunnels** to move laterally.

How to Detect Lateral Movement

1. **Monitor Network Traffic**:

- Look for **unusual outbound connections** to unexpected IPs or domains.
- Use **SIEM tools** (e.g., Splunk, ELK Stack) to flag anomalies in traffic patterns.

2. **Analyze User Behavior**:

- Detect **suspicious login times**, repeated failed login

attempts, or logins from unusual locations.

- Use **user and entity behavior analytics (UEBA)** to identify deviations from normal activity.

3. **Check for Credential Compromise**:

- Monitor for **hashes or tickets** in memory (e.g., via **Mimikatz** or **bloodhound**).

- Use **endpoint detection and response (EDR)** tools (e.g., CrowdStrike, SentinelOne) to detect credential theft.

4. **Audit System Logs**:

- Review event logs for **failed login attempts**, **unauthorized access**, or **unexpected process executions**.

- Look for **SMB/HTTP/SSH traffic** to systems not in the user's normal activity.

5. **Implement Zero Trust**:

- Enforce **least privilege** and **multi-factor authentication (MFA)**.

- Use **network segmentation** to limit lateral movement.

Example Detection Signs

- A user logs in from a non-trusted IP and immediately connects to a system they don't normally access.

- A system shows **unusual SMB or RDP connections** to internal resources.

- **Kerberos tickets** are found in memory or on the network (e.g., via **bloodhound** or **klist**).

Action: Isolate affected systems, investigate logs, and remediate credentials or vulnerabilities.

Let me know if you need tools or MITRE ATT&CK framework details.

```

In [50]: def launch_chat_widget():
out      = widgets.Output()
text_in  = widgets.Text(placeholder='Ask about threats, CVEs, incident
response...',
                        layout=widgets.Layout(width='75%'))
send_btn = widgets.Button(description='Ask', button_style='primary',
                        layout=widgets.Layout(width='10%'))
clear_btn = widgets.Button(description='Clear', button_style='warning',
                        layout=widgets.Layout(width='10%'))

def on_send(b):
    q = text_in.value.strip()
    if not q: return
    text_in.value = ''
    with out:
        print(f"\nUSER: {q}")
        ans = threat_chat(q)
        print(f"AI: {ans}")
        print("-"*60)

def on_clear(b):
    global _chat_history
    _chat_history = []
    out.clear_output()
    with out:
        print("Chat cleared ")

send_btn.on_click(on_send)
clear_btn.on_click(on_clear)
text_in.on_submit(on_send)

display(widgets.VBox([
    widgets.HTML("<h3 style='color:#00d4ff;font-family:monospace'>🛡️
CyberShield AI Chat</h3>"),
    widgets.HBox([text_in, send_btn, clear_btn]),
    out
]))

launch_chat_widget()

```

```

VBox(children=(HTML(value="<h3 style='color:#00d4ff;font-
family:monospace'>🛡️ CyberShield AI Chat</h3>"), HBox(...

```

In []:

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).